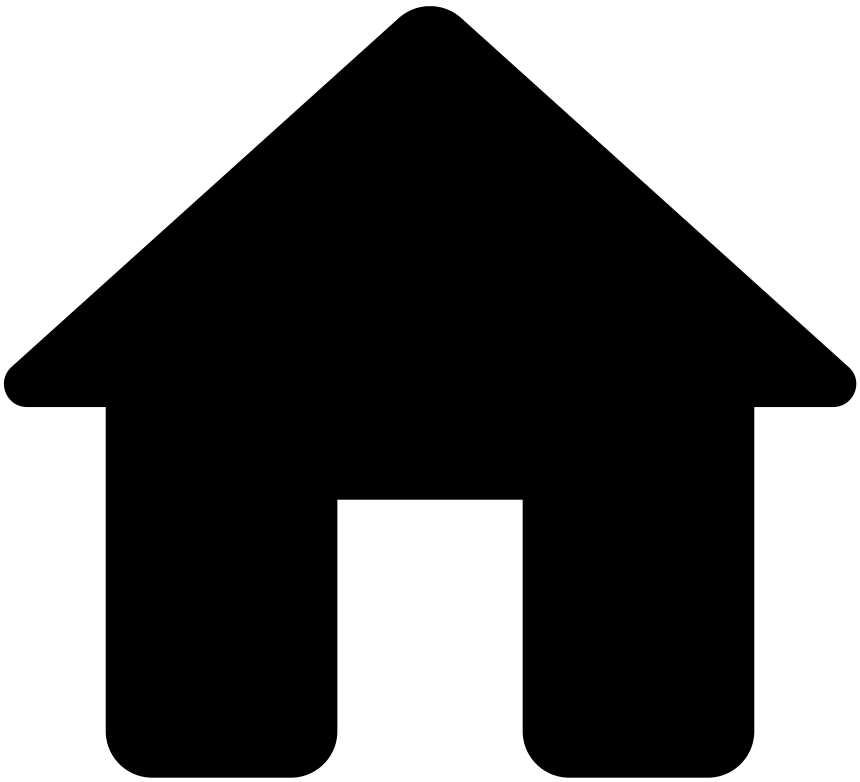


Generating the Features Dataset

Typically, you want to train your machine learning model on a data source that contains derived fields and metrics. You can achieve this by building up your feature plan(s) and generating a feature data source.

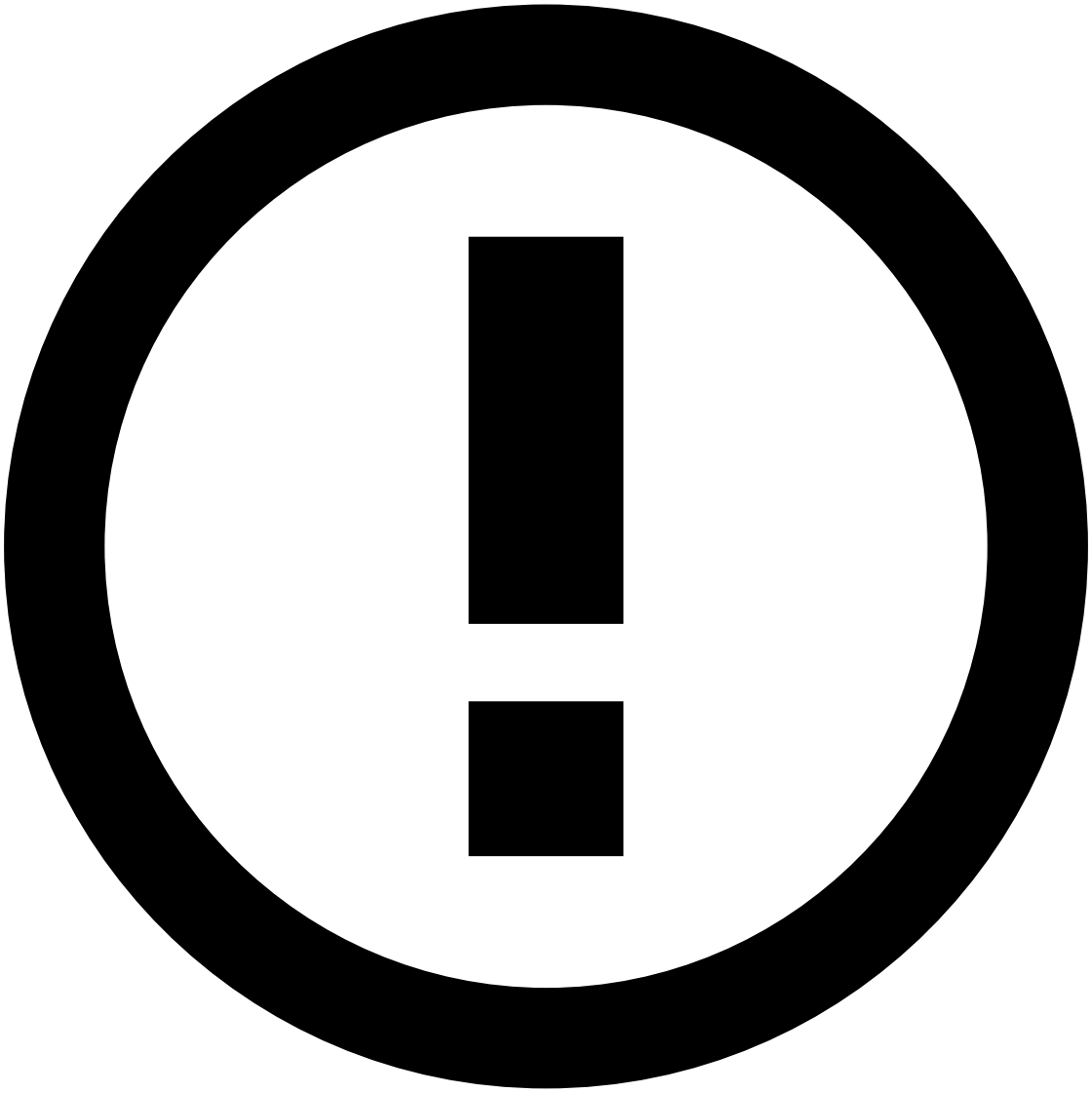
This page gives you instructions on how to create such a data source.

Preparing the planâ□□

To produce your feature dataset, compose a plan as described in the following pages:

- Designing derived features

- Designing metrics (historical profiles)

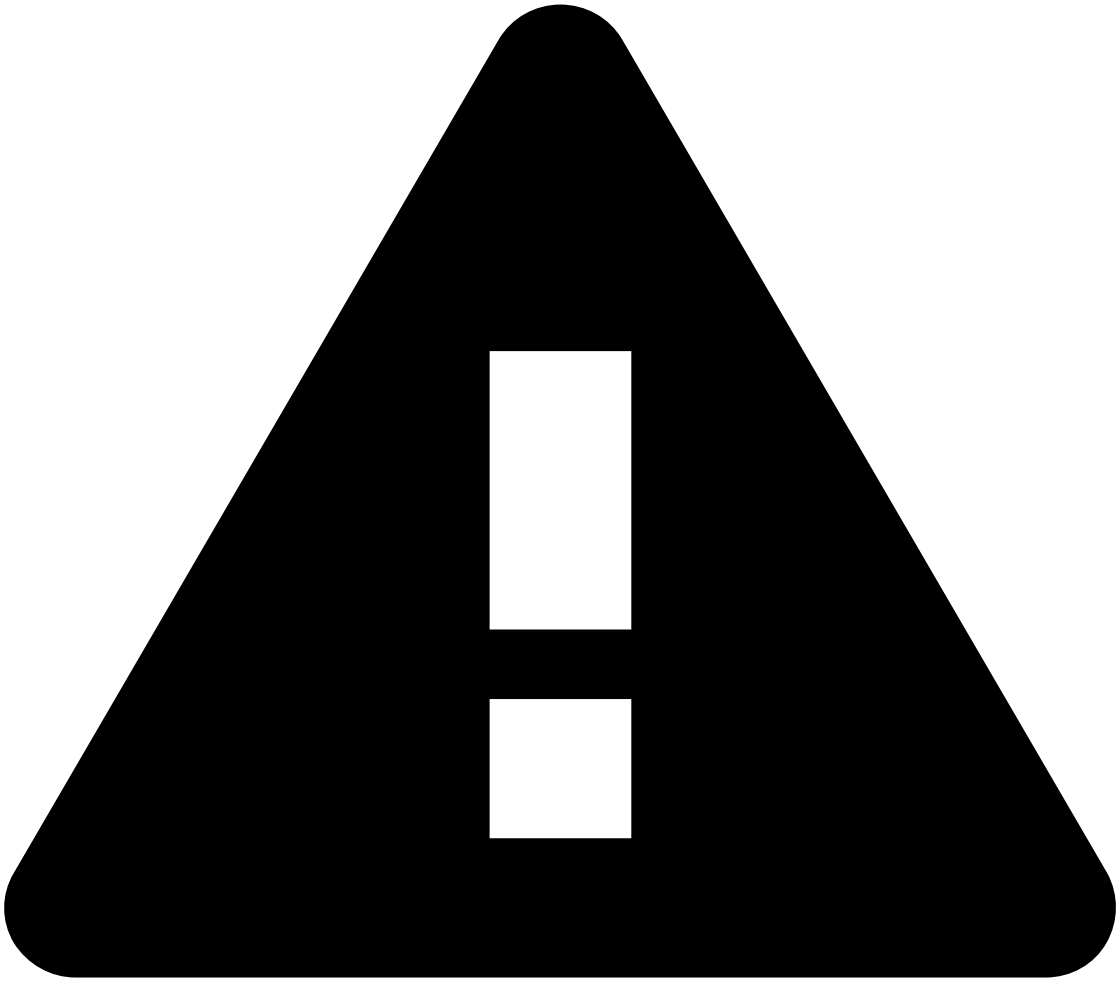


Target field

Real-time data, coming into Pulse in Production, will not include the target field you aim at predicting during the data science stage. In fact, in a workflow, you configure that target as an outcome that represents Pulse's decision for each instance.

Before deploying your feature plan to runtime, you may want to:

1. Edit the map operator to remove the target field from the plan's output data source.
2. Export the output data source to generate a schema without the target field.

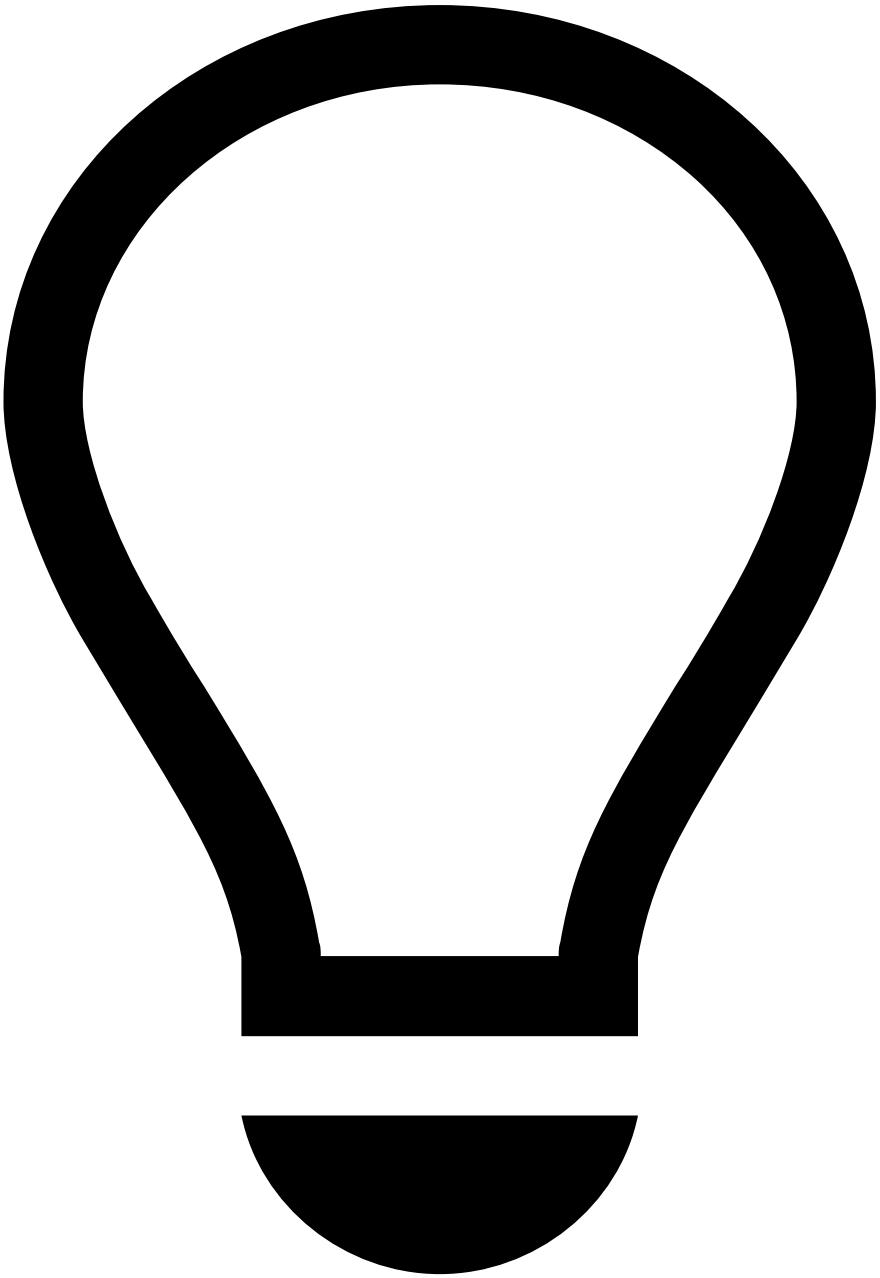


Field dependencies

An operator in a plan that can add fields to the schema cannot have intra-operator dependencies. In map and metrics operators, you cannot refer to a field that has been added inside that same operator.

Running the plan to generate the enhanced data sourceâ□□

1. Use the run button to execute the plan.
    1. In the **Data Source** tab, select the data source that will be the input for the plan. For example *Cleaned Transactions*. If the plan has more than one channel, select the input data source for each channel.
    2. In the **Data Sinks** tab, configure the output of the plan:
        - **Name** for the output data source. For example, *Features v1*.
        - **Type**: the file format of the output data â□□*Parquet* (default) or *CSV*.



Parquet format is recommended

Using the Parquet file format for data files is highly recommended. Besides allowing a direct encoding of data types (which automates the mapping of field types when importing datasets), it also improves performance and disk usage: compared to CSV, Parquet files occupy less space and take less time to load.

- **Directory Path**: the path and file name where Pulse will store the output CSV file, containing the data of the new data source. Example: `<file:///datasets/rule-features-v1.parquet>`. Note that the format varies according to the operating system.

3. Use the run button to execute the plan. It generates a new data source in Pulse, named *Features v1*. Pulse also saves an output file with the transformed data to your filesystem.

2. Check the new data source to confirm that its schema includes new fields.

Designing the final feature datasetâ□□

There are multiple approaches when using plans to obtain your final feature dataset. The one you choose comes down to how it best fits your requirements and work methodology.

Single plan with one mapâ□□

The most straightforward approach is encapsulating all feature engineering logic into one map operator. This is the simplest approach and the one proposed in the current topic.

The following image depicts a simple features plan where all features are designed in the metrics operator and a single map operator.

Advantagesâ□□

- Straightforward

Disadvantagesâ□□

- Hard to maintain and reuse. With the number of features easily rising to hundreds, it is very hard to keep track of all features, group them by type, and generally keep them clean and organized in the same map.

Sequential plansâ□□

A more segmented approach consists in using isolated plans to sequentially manipulate data, add features and metrics to datasets. In this case, you have to manually run each plan and manage how all intermediate data sources are consumed and produced.

For example, having separate plans to design metrics (historical profiles) and to design derived features, that you manually run one after the other.

The following image depicts a composition of four separate plans. Each plan generates different features/historical profiles, taking as input the data source that the previous plan produced.

Advantagesâ□□

- Feature engineering logic is isolated by plan, improving maintainability and reuse.

Disadvantagesâ□□

- Hard and time-consuming to execute and reproduce the entire sequence. You have to manually execute each intermediate step, such as running each plan. You also must be extremely careful to ensure that you chain them together with the correct data sources to be consumed and produced.

Single plan with several mapsâ□□

Another possibility, which Feedzai **recommends**, is using a single plan that contains all Feature Engineering logic split among several map operators. Each map adds different types of features to the dataset, effectively isolating logic while keeping complexity under control.

The following image displays an example of a features Plan where all features are grouped by type. Note that auxiliary fields are removed in the end and that all features that do not depend on historical profiles are designed before the metrics operator.

Advantagesâ□□

- All feature engineering logic is contained in the same plan, simplifying the process. There is no need for manual, intermediate, actions and datasets.
- Feature engineering logic is isolated by map, improving maintainability and reuse for each type of features. Even when the number of features becomes too large, they are still grouped by type. This allows you to:
    - Easily keep track of related features.
    - Know where each type of features is being generated.
    - Work on each type of features at a time and iteratively test and build up the plan.
    - Easily detect possible dependencies among features. For example, time-based features are derived from historical profiles.

Disadvantagesâ□□

- The plan, although linear, becomes more complex with a lot more steps.